



Smart Infrastructure Protection and Continuity System

Distributed Data Plane Redirection Using VIP Technology

Author	Ali Yasser Ali Mohamed
Supervisor	Dr. Hisham Zarif

Abstract

High-availability systems are traditionally designed to react to complete node failures, where an entire server becomes unreachable. However, in many real-world scenarios, a server may remain operational at the operating system level while a critical service running on it (such as an HTTP server) fails. In such cases, treating the entire node as failed is inefficient and may lead to unnecessary service disruption. This report presents a detailed design and implementation of a service-level failover mechanism using a Dynamic Virtual IP (VIP) controlled by a centralized controller. The approach ensures that traffic is redirected to a backup server only when the certain service fails. The solution relies on Keepalived, VRRP, and a custom health-check script that continuously validates the service. All technical steps, configurations, and scripts are explained in detail.

1. Introduction

High availability (HA) is a core requirement in modern networked systems. Many HA designs operate at the node level, meaning that failover is triggered only when a server becomes completely unreachable. While this approach is simple, it does not address situations where a server is alive but a specific service is malfunctioning.

In this report, we focus on service-level availability, specifically for an HTTP service. The goal is to ensure that client devices continue to access the HTTP service even if the service fails on the active server, without requiring any changes on the client side and without routing traffic through a centralized proxy.

The key idea is to: - Represent the HTTP service using a Virtual IP (VIP) - Dynamically move this VIP between servers - Trigger failover only when the HTTP service is unhealthy

This design maintains a strict separation between the control plane (centralized decision-making) and the data plane (direct client-to-server communication), which aligns well with modern SDN and carrier-grade protection principles.

2. System Architecture and Roles

The system consists of the following components:

- **A:** Original HTTP server (failed)
- **B:** Backup servers capable of running the HTTP service
- **C:** Centralized controller (admin node)
- **X:** Client host that depends on the HTTP service

Key Architectural Principles:

- Clients never connect to a physical server IP.
- Clients always connect to a **service IP (VIP)**.
- The VIP is owned by exactly one server at a time.
- The controller decides which server should own the VIP.
- Data traffic never passes through the controller.

3. Tools and Technologies

3.1 Keepalived

Keepalived is a Linux-based service that implements the Virtual Router Redundancy Protocol (VRRP). It allows multiple machines to share a single virtual IP address while ensuring that only one machine actively owns it at any time.

In this design, Keepalived is used to: - Assign the VIP to the active HTTP server - Periodically execute a health-check script - Remove the VIP if the health check fails

3.2 VRRP (Virtual Router Redundancy Protocol)

VRRP is a protocol that enables multiple nodes to cooperate in presenting a single virtual IP. One node acts as the **MASTER**, while others remain **BACKUP**. In this implementation, VRRP behavior is tightly controlled by the admin node to ensure deterministic placement.

4. Virtual IP Design

A single virtual IP is allocated to represent the HTTP service:

VIP = 10.0.0.100

This IP is: - Known to all clients - Never permanently assigned to any server interface - Dynamically attached and detached by Keepalived

Clients always send requests to http://10.0.0.100, regardless of which server currently hosts the service.

5. HTTP Service Health-Check Script

5.1 Purpose of the Script

The health-check script determines whether the HTTP service is healthy enough for the server to continue owning the VIP. Keepalived interprets the script result strictly:

- Exit code 0: Service is healthy → keep VIP
- Non-zero exit code: Service is unhealthy → remove VIP

5.2 Script Location

`/usr/local/bin/check_httpd.sh`

5.3 Full Script Code

```
#!/bin/bash

# Check if httpd process is running
if ! pgrep httpd > /dev/null; then
    exit 1
fi

# Check if HTTP responds correctly
HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1)

if [ "$HTTP_STATUS" != "200" ]; then
    exit 1
fi

exit 0
```

5.4 Script Explanation

1. **Shebang (`#!/bin/bash`)**
 - Specifies that the script must be executed using Bash.
2. **Process Check (`pgrep httpd`)**
 - Verifies that the HTTP daemon is running.
 - If the process does not exist, the script fails immediately.
3. **HTTP Response Check (`curl`)**
 - Sends an HTTP request to 127.0.0.1.
 - Discards the response body.
 - Extracts only the HTTP status code.
4. **Status Validation**
 - Ensures the service responds with 200 OK.
 - Any other status code is treated as a failure.
5. **Exit Code**
 - `exit 0` confirms service health.
 - `exit 1` triggers VIP removal.

This two-stage validation prevents false positives where the process exists but the service is not functional.

6. Keepalived Configuration

6.1 Health-Check Definition

```
vrp_script chk_http {  
    script "/usr/local/bin/check_httpd.sh"  
    interval 2  
    timeout 1  
    fall 2  
    rise 1  
}
```

- interval 2: Script runs every 2 seconds
- fall 2: Two consecutive failures trigger failover
- rise 1: One success restores health

6.2 VRRP Instance Configuration

```
vrp_instance HTTP_VIP {  
    state MASTER  
    interface eth0  
    virtual_router_id 51  
    priority 150  
    advert_int 1  
  
    track_script {  
        chk_http  
    }  
  
    virtual_ipaddress {  
        10.0.0.100  
    }  
}
```

This configuration ensures that: - The VIP is attached only when the HTTP service is healthy - VIP ownership is immediately revoked on service failure

- **virtual_ipaddress:** must be within the ip pool range of the network
EX: if network ID is 10.0.0.0 and hosts acquire Ips 10.0.0.2 – 10.0.0.99, so VIP should be a number between 10.0.0.100 – 10.0.0.254
- **virtual_router_id:** all machines that share VIP must have same virtual router ID
- **Priority:** higher priority makes the machine is the master, backup servers must have lower priority than master
- **State:** MASTER/BACKUP all backup servers must have the same keepalived.conf file but with modifying the state into BACKUP and modify priority (lower)

6.3 Full keepalived.conf Script

```
vrp_script chk_http {
    script "/usr/local/bin/check_httpd.sh"
    interval 2
    timeout 1
    fall 2
    rise 1
}

vrp_instance HTTP_VIP {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 150
    advert_int 1

    track_script {
        chk_http
    }

    virtual_ipaddress {
        10.0.0.100
    }
}
```

7. Failure and Recovery Workflow

1. HTTP service is running normally on the active server.
2. Keepalived continuously executes the health-check script.
3. HTTP service fails (process stops or stops responding).
4. Health-check script returns failure.
5. Keepalived removes the VIP from the server.
6. The controller selects a new backup server.
7. HTTP service is deployed on the selected server.
8. Keepalived is activated on the new server.
9. The VIP is announced using gratuitous ARP.
10. Clients automatically reconnect to the new server.

Throughout this process, clients are never reconfigured and never communicate with the controller directly.

8. Advantages of the Approach

- Service-level protection instead of node-level protection
- No client-side configuration changes
- No centralized traffic forwarding
- Fast convergence (seconds)
- Clear separation of control and data planes

9. Conclusion

This report presented a complete and detailed implementation of a service-level HTTP failover mechanism using a Dynamic Virtual IP. By combining Keepalived, VRRP, and a custom HTTP health-check script, the system reacts precisely to HTTP service failures without overreacting to partial node issues. The approach achieves high availability while maintaining architectural simplicity, scalability, and strong alignment with modern networking principles such as SDN-inspired control-plane separation and fast protection switching.

10. References

No.	Title	Source	URL
1	Keepalived Official Project Page	Keepalived	https://keepalived.org
2	Virtual Router Redundancy Protocol (VRRP)	IETF / RFC Editor	https://www.rfc-editor.org
3	Keepalived —Failover using VRRP	Keepalived	https://keepalived.org/doc/
4	A High Availability Clusters Model Combined with Load Balancing and Shared Storage Technologies for Web Servers	arXiv	https://arxiv.org/abs/1411.7658
5	Hybrid load balancing method with failover capability in server cluster using SDN	arXiv	https://arxiv.org/abs/2307.05552